

Documentație 6

Implementarea Releului și
Logica Software pentru
Controlul cu Histereză

Student: Șișca Silviu-Claudiu

Anul: III, 2133

Cuprins

1. Cerința Proiectului.....	3
2. Implementarea sistemului de comandă.....	4
3. Logica de comandă a butoanelor	6
3.1. Stabilizatorul liniar.....	6
4. Interpretarea Datelor	9
5. Afișarea Datelor pe LCD.....	12
6. Simulări.....	Error! Bookmark not defined.
7. Cod C & Assembly.....	16
8. Concluzii.....	Error! Bookmark not defined.

1. Cerința Proiectului

Obiectivul acestui proiect este proiectarea și implementarea unui sistem de comandă cu releu, care este controlat de un microcontroler 8051. Microcontrolerul trebuie programat în C, respectiv Assembly, având la bază codul din tema trecută.

Capabilitățile pe care trebuie să le îndeplinească sistemul sunt:

- Modificarea temperaturii de referință prin intermediul unor butoane.
- Comanda releului prin intermediul microcontrolerului.
- Implementarea unui algoritm de control cu histereză pentru comutarea releului.

2. Implementarea sistemului de comandă

Având în vedere faptul că, pentru acționarea unui releu standard este necesar un curent de aproximativ 60-80 mA, comanda releului electromagnetic din circuit nu se poate realiza direct din portul microcontrolerului 8051, deoarece capacitatea în curent a acestuia este cu mult depășită. Prin urmare, a fost necesară proiectarea unui etaj intermediar de putere, bazat pe un tranzistor NPN, configurat ca întrerupător comandat pe partea de masă (Low-Side Switch).

Implementarea circuitului este ilustrată în Figura 1. Așa cum se poate observa în schemă, pentru a asigura o comutație sigură și pentru a proteja microcontrolerul, circuitul integrează o serie de componente active și pasive:

- **Tranzistorul de comutație - Q2:** acționează ca un întrerupător electronic. Când pinul microcontrolerului furnizează un semnal logic HIGH, tranzistorul intră în regiunea de saturație, permițând trecerea curentului prin bobina releului către masă și închizând astfel contactul mecanic.
- **Rezistența de bază - R13:** are rolul de a limita curentul extras din pinul microcontrolerului. Curentul de bază injectat în tranzistor se calculează aplicând legea lui Ohm, considerând tensiunea de ieșire a portului 5V și căderea pe joncțiunea bază-emitor 0.7V.

$$I_B = \frac{5V - 0.7V}{1000\Omega} = 4.3mA$$

Această valoare este sigură pentru portul microcontrolerului și suficientă pentru a asigura saturația tranzistorului Q2.

- **Dioda de protecție - D3:** cunoscută sub denumirea de diodă de regim liber, aceasta este conectată antiparalel cu bobina releului. Deoarece bobina reprezintă o sarcină puternic inductivă, în momentul trecerii tranzistorului Q2 în starea de blocare, câmpul magnetic colapsează brusc, generând un vârf de tensiune autoindusă extrem de mare. Dioda D3 oferă o cale sigură de descărcare a acestui curent tranzitoriu, prevenind distrugerea circuitului.
- **Circuitul de sarcină simulată – LED D2 și R12:** pentru a valida vizual închiderea contactului al releului, a fost implementat un circuit de semnalizare. La cuplarea releului, tensiunea de 5V alimentează un LED roșu. Rezistența R12 limitează curentul prin LED la o valoare de aproximativ 10mA, având în vedere căderea de tensiune tipică pe un LED de această culoare este de 2V.

$$I_{LED} = \frac{5V - 2V}{300\Omega} = 10mA$$

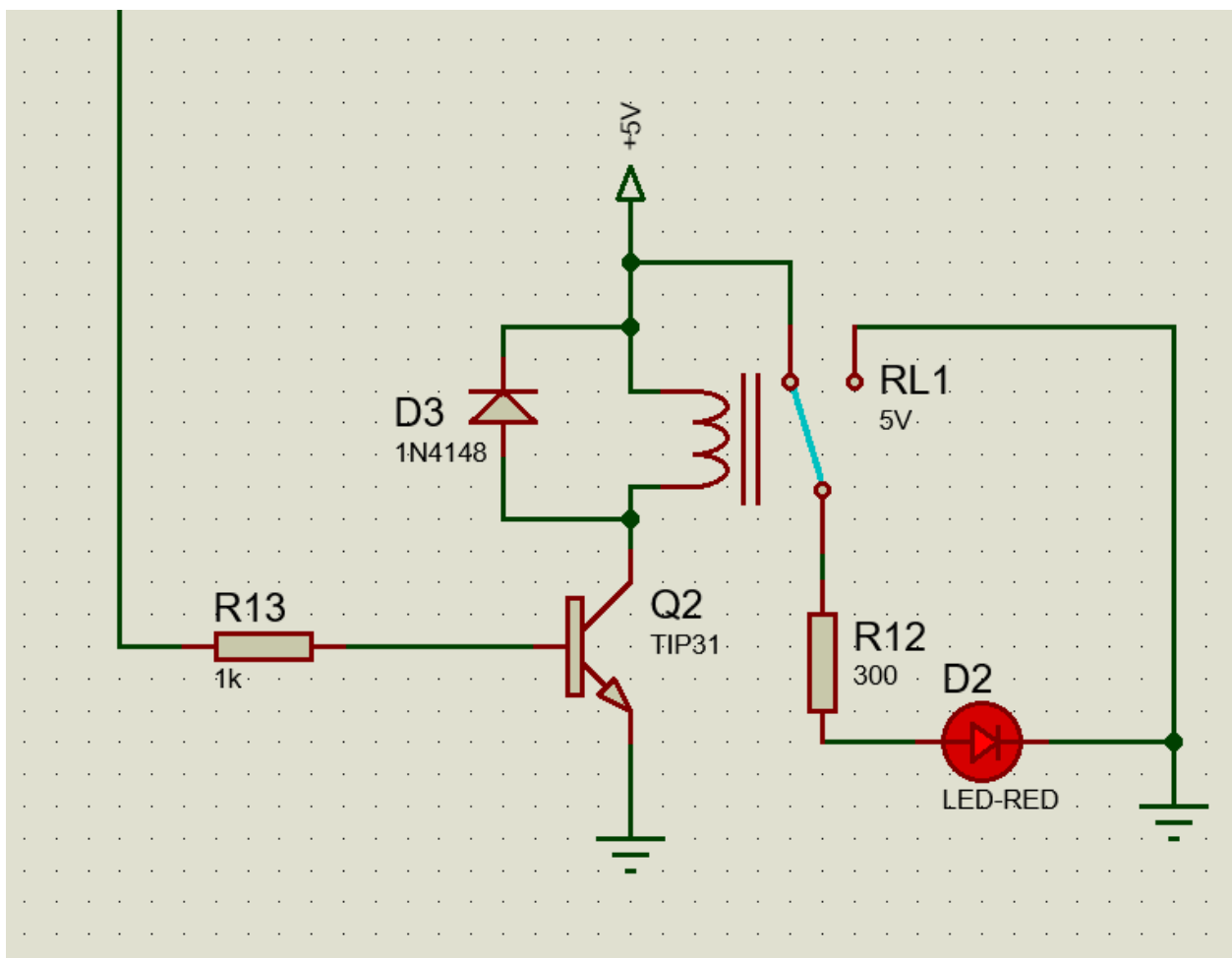


Figura 1: Circuitul de comandă

3. Logica de comandă a butoanelor

Pentru a permite utilizatorului ajustarea temperaturii de referință, au fost implementate două butoane ‘Push-Buttons’. Implementarea hardware a acestora este prezentată în Figura 2.

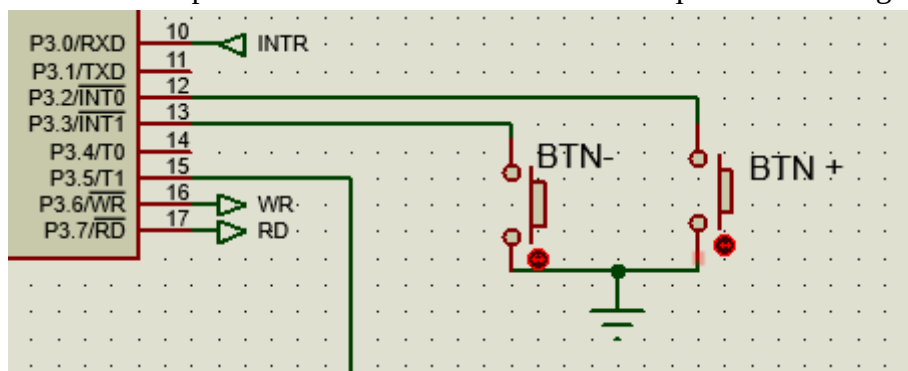


Figura 2: Implementarea butoanelor

3.1. Conexiunea hardware

Butoanele sunt conectate la pinii *P3.2* și *P3.3* ai microcontrolerului 8051, deoarece acești pini corespund intrărilor pentru întreruperile externe hardware *INT0* și *INT1*.

Din punct de vedere electric, butoanele sunt configurate în modul *Active-Low*. Pinii portului 3 dispun de rezistențe interne de pull-up, menținând un nivel logic HIGH în starea de repaus. La apăsarea butonului, circuitul se închide la masă, aducând pinul în starea logică LOW și declanșând rutina de tratare a întreruperii.

3.2. Implementarea Software

Din cauza prezenței impulsurilor parazite la apăsarea sau eliberarea butonului, a fost necesară implementarea unui algoritm de *Debounce*.

Algoritmul funcționează în felul următor: după declanșarea întreruperii, execuția este suspendată pentru 20 ms, apoi se citește din nou starea pinului pentru a valida că butonul încă este apăsat. După modificarea valorii de referință, algoritmul blochează execuția într-o buclă de așteptare până la eliberarea completă butonului, urmată de o nouă suspendare a execuției pentru 20ms pentru a filtra zgomotul de decuplare.

Pentru a asigura o funcționare sigură a termostatlui și pentru a preveni setarea unor valori anormale, au fost impuse limite stricte pentru temperatura de referință: un minim de 10.0°C și un maxim de 30.0°C.

Mai jos este prezentată implementarea logicii pentru tratarea întreruperii generate de apăsarea butonului “BTN+”, realizată comparativ în limbajele C și Assembly.

În limbajul C, protecția se realizează simplu, printr-o condiție logică multiplă evaluată direct în interiorul rutinei de tratare a întreruperii externe.

```

void plus() interrupt 0{
    delay(20);
    if(BTN_PLUS == 0 && set_value < max_value)
    {
        set_value += 5;
        update_lcd_flag = 1;
    }
    while(BTN_PLUS == 0);
    delay(20);
}

```

În limbajul Assembly, deoarece se operează strict pe 8 biți, compararea valorii curente pe 16 biți cu limita de 300 se realizează manual, printr-o scădere cu împrumut.

```

ISR_PLUS:
    PUSH PSW
    PUSH ACC
    PUSH B

    MOV A, #14h
    MOV B, #00h
    ACALL DELAY

    JB BTN_PLUS, EXIT_PLUS

    CLR C
    MOV A, SET_VAL_L
    SUBB A, #2Ch
    MOV A, SET_VAL_H
    SUBB A, #01h
    JNC WAIT_RELEASE_PLUS

    MOV A, SET_VAL_L
    ADD A, #05h
    MOV SET_VAL_L, A
    MOV A, SET_VAL_H
    ADDC A, #00h
    MOV SET_VAL_H, A

    SETB FLAG_LCD

    WAIT_RELEASE_PLUS: JNB BTN_PLUS, WAIT_RELEASE_PLUS

    MOV A, #14h
    MOV B, #00h
    ACALL DELAY

    EXIT_PLUS:
    POP B
    POP ACC

```

3.3. Gestiunea întreruperilor

Comunicare microcontrolerului cu afișajul LCD presupune respectarea unor diagrame de timp stricte pentru transmiterea comenzilor și a datelor. Pentru a preveni coruperea informației afișate, un exemplu este prezentat în Figura 3, pe durata execuției blocurilor de cod responsabile cu afișarea temperaturii, întreruperile globale sunt dezactivate temporar. Această măsură de siguranță garantează că o eventuală apăsare de buton nu va suspenda și desincroniza protocolul de comunicație cu LCD-ul. Imediat după terminarea transmisiei datelor către ecran, întreruperile sunt reactivate, permițând sistemului să răspundă din nou la comenzile utilizatorului.

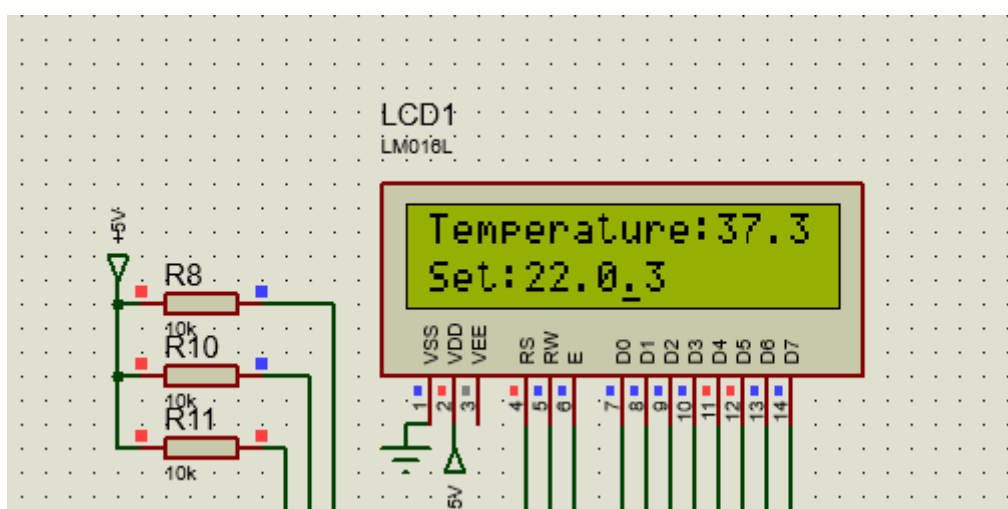


Figura 3: Exemplu de corupere a datelor

4. Algoritmul de control si implementarea histerezei

Pentru implementarea algoritmului de decizie cu histereză, s-a definit o marjă de $\pm 0.5^{\circ}\text{C}$ în jurul valorii setate, aceasta fiind reprezentată de valoarea matematică 5 în logica microcontrolerului. Astfel, releul va cupla doar când temperatura scade sub limita inferioară și se va decupla doar când temperatura depășește limita superioară. Cât timp temperatura se află în interiorul acestui interval, starea releului rămâne neschimbată, asigurând o funcționare lină a elementului de încălzire.

În C, implementarea histerezei este directă și intuitivă. Funcția 'process_data' primește valoarea de la convertorul analog-digital, o convertește în temperatura reală, iar apoi utilizează o structură decizională simplă de tip 'if-else if' pentru a comanda pinul releului.

```
void process_data(unsigned char val){  
  
    temp_value = ((unsigned int)val * 80 ) / 51;  
  
    if(temp_value < (set_value - 5)){  
        RELAY = 1;  
    }  
    else if( temp_value > (set_value + 5)){  
        RELAY = 0;  
    }  
  
    show_value(temp_value);  
}
```

În Assembly, arhitectura pe 8 biți necesită o abordare mai complexă. Compararea valorii curente cu limitele calculate presupune efectuarea unei operații de scădere pe 16 biți cu împrumut. Dacă operația de scădere generează un Flag de Carry, înseamnă că valoarea curentă este strict mai mică decât limita cu care este comparată. Pentru a nu distruge datele originale de pe ecran, limitele sunt calculate și stocate temporar în regiștrii R4 și R5.

```
PROCESS_DATA:  
    MOV B, #50h  
    MUL AB  
    MOV R0, B  
    MOV R1, A  
  
    MOV R2, #00h  
    MOV R3, #00h  
  
DIV_51:  
    CLR C  
  
    MOV A, R1  
    SUBB A, #33h  
    MOV R1, A
```

```

MOV A, R0
SUBB A, #00h
MOV R0, A

JC DIV_DONE
INC R3
MOV A, R3
JNZ DIV_51
INC R2
JMP DIV_51
DIV_DONE:
MOV TEMP_VAL_H, R2
MOV R0, TEMP_VAL_H
MOV TEMP_VAL_L, R3
MOV R1, TEMP_VAL_L

VERIFY_1:
CLR C
MOV A, SET_VAL_L
SUBB A, #05h
MOV R4, A
MOV A, SET_VAL_H
SUBB A, #00h
MOV R5, A

CLR C
MOV A, TEMP_VAL_L
SUBB A, R4
MOV A, TEMP_VAL_H
SUBB A, R5
JNC VERIFY_2

SETB RELAY
SJMP FINISH
VERIFY_2:
MOV A, SET_VAL_L
ADD A, #05h
MOV R4, A
MOV A, SET_VAL_H
ADDC A, #00h
MOV R5, A

CLR C
MOV A, TEMP_VAL_L
SUBB A, R4
MOV A, TEMP_VAL_H
SUBB A, R5
JC FINISH

CLR RELAY

```

```
    FINISH:
      ACALL SHOW_VALUE
RET
```

5. Simulări

Pentru a demonstra funcționalitatea circuitului implementat, au fost realizate mai multe simulări. În secțiunea următoare sunt prezentate capturi de ecran ce surprind răspunsul sistemului în cele mai importante scenarii de funcționare.

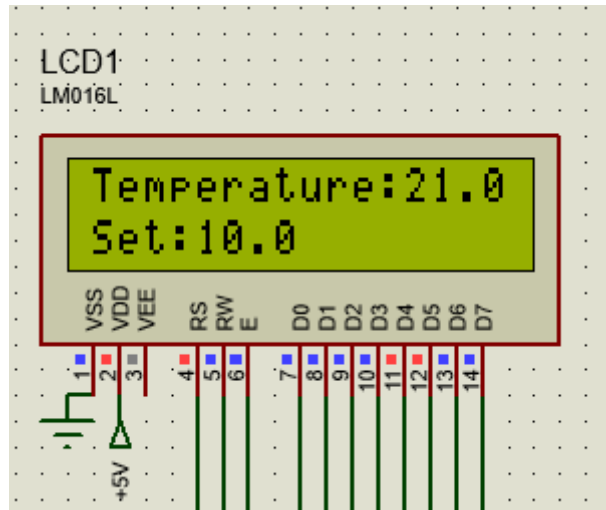


Figura 4: Temperatura de referință minimă

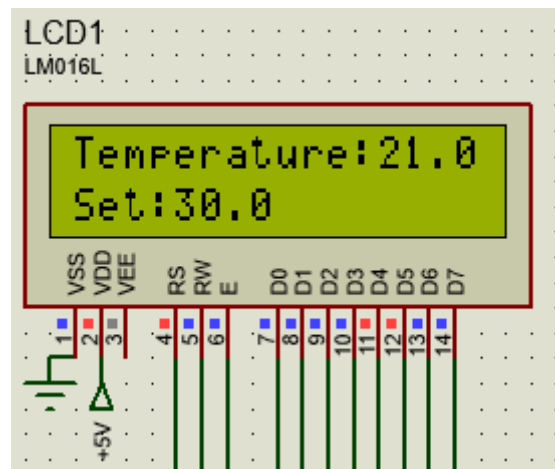


Figura 5: Temperatura de referință maximă

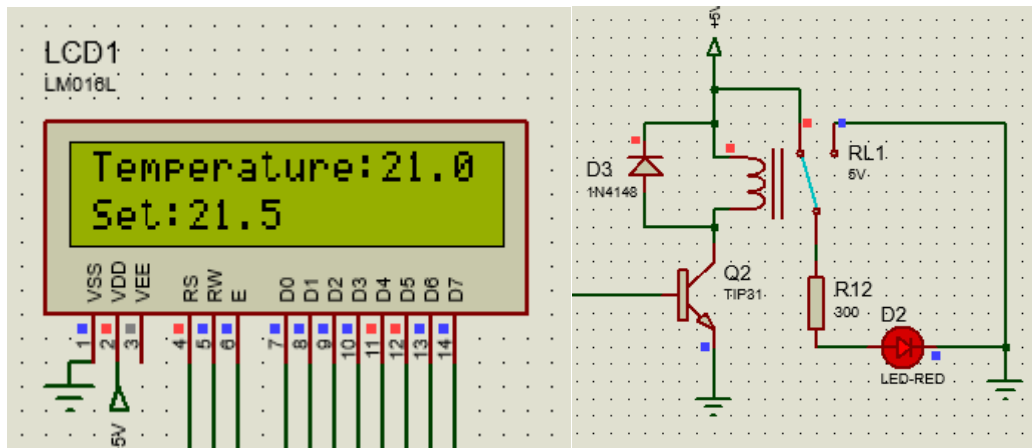


Figura 6: Temperatura de start & Răspunsul sistemului

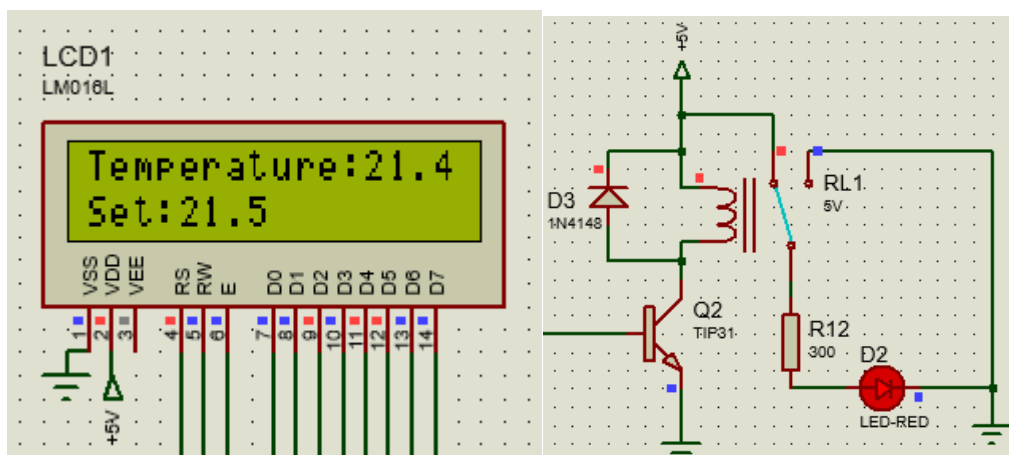


Figura 7: Temperatura intermediară & Răspunsul sistemului

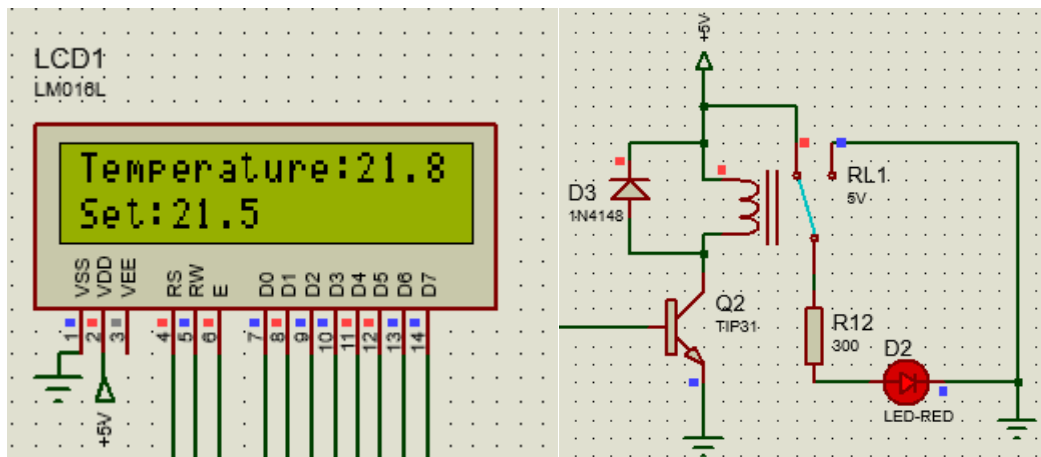


Figura 8: Temperatura intermediară & Răspunsul sistemului

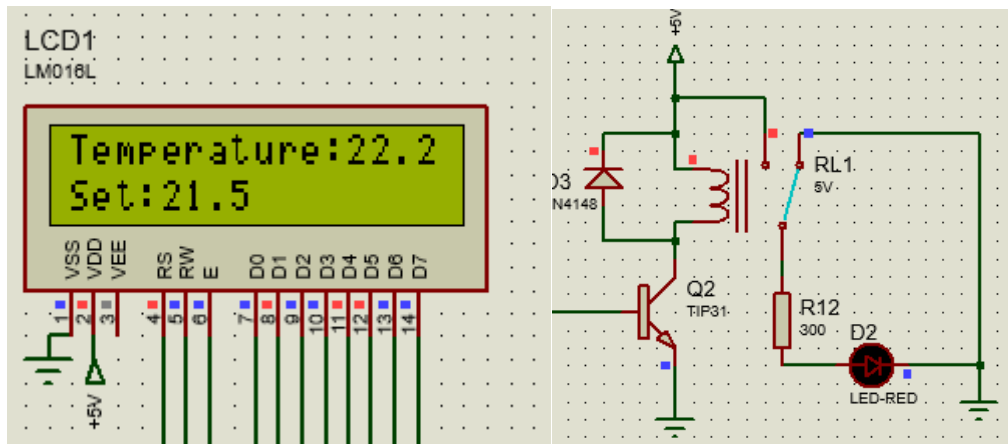


Figura 9: Temperatura intermediară & Răspunsul sistemului

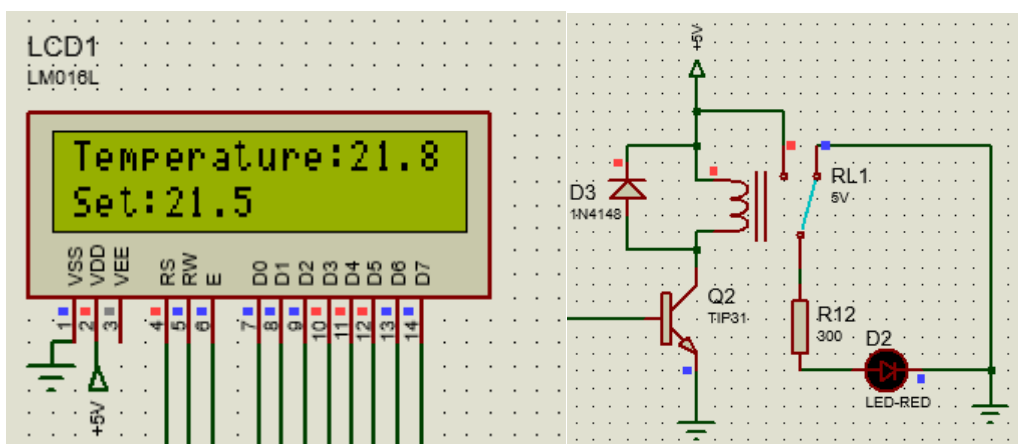


Figura 10: Temperatura intermediară & Răspunsul sistemului

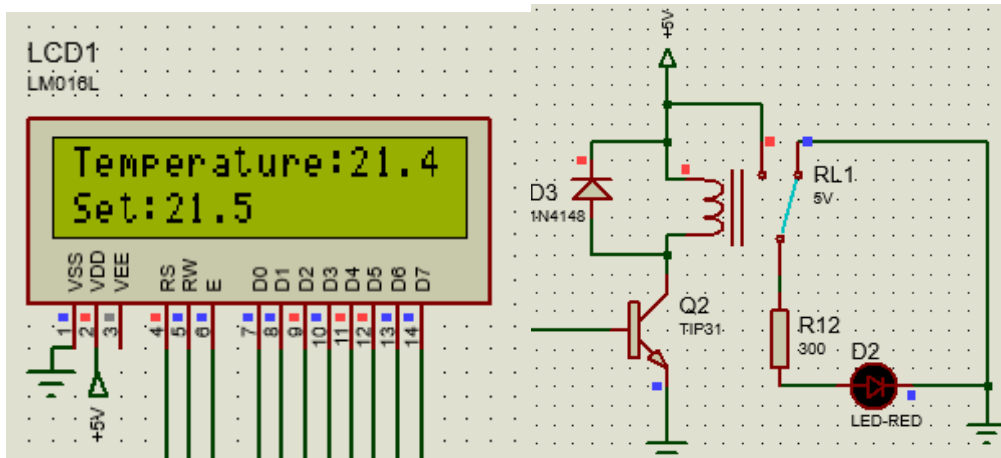


Figura 11: Temperatura intermediară & Răspunsul sistemului

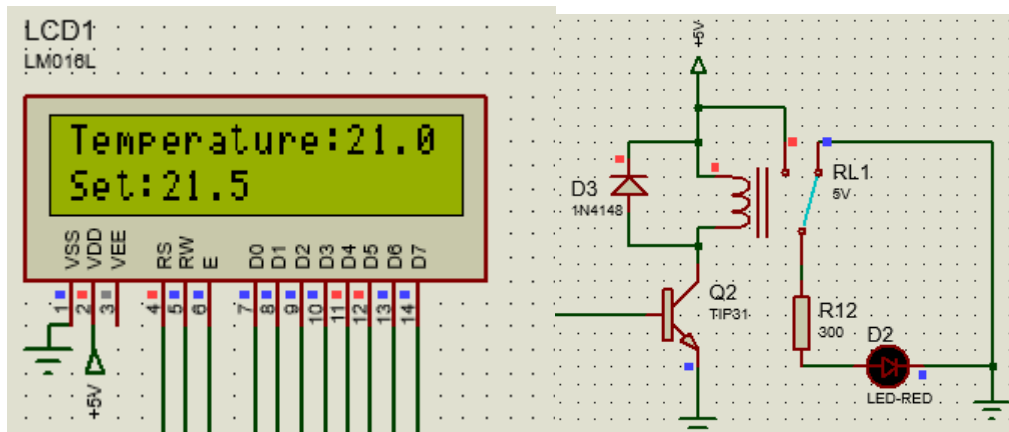


Figura 12: Temperatura intermediară & Răspunsul sistemului

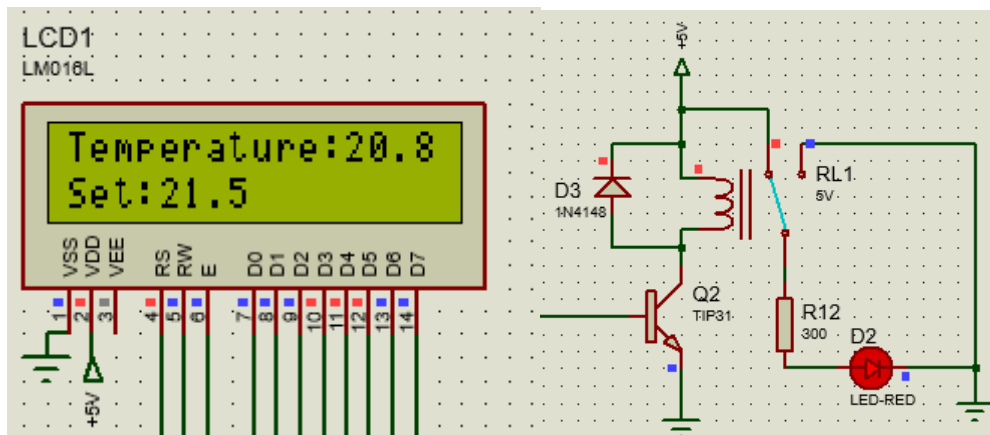


Figura 13: Temperatura intermediară & Răspunsul sistemului

7. Cod C & Assembly

```
#include <REG51.H>

#define ADC_DATA P1
#define LCD P2

sbit RS = P0^0;
sbit RW = P0^1;
sbit E = P0^2;
sbit INTR = P3^0;
sbit BTN_PLUS = P3^2;
sbit BTN_MINUS = P3^3;
sbit RELAY = P3^5;
sbit WR_ADC = P3^6;
sbit RD_ADC = P3^7;

unsigned char code string[] = "Temperature:";
unsigned char code set[] = "Set:";
unsigned int set_value = 210;
unsigned int max_value = 300;
unsigned int min_value = 100;
unsigned int temp_value;
bit update_lcd_flag = 0;

void delay(int ms);
void command(int hex);
void setup_lcd();
void show_char(unsigned char x);
void show_value(unsigned int val);
void process_data(unsigned char val);
unsigned char read_adc();

void main(){
    ADC_DATA = 0xFF;
    LCD = 0;
    RW = 0;
    RELAY = 0;
    INTR = 1;
    BTN_PLUS = 1;
    BTN_MINUS = 1;
    IT0 = 1;
    IT1 = 1;
    IE = 0x85;
    setup_lcd();

    while(1){
        EA = 0;
        command(0x8C);
```



```

    process_data(read_adc());
    EA = 1;

    if(update_lcd_flag){
        EA = 0;
        command(0xC4);
        show_value(set_value);
        EA = 1;
        update_lcd_flag = 0;
    }

    delay(300);
}

void plus() interrupt 0{
    delay(20);
    if(BTN_PLUS == 0 && set_value < max_value)
    {
        set_value += 5;
        update_lcd_flag = 1;
    }
    while(BTN_PLUS == 0);
    delay(20);
}

void minus() interrupt 2{
    delay(20);
    if(BTN_MINUS == 0 && set_value > min_value){
        set_value -= 5;
        update_lcd_flag = 1;
    }
    while(BTN_MINUS == 0);
    delay(20);
}

void delay(int ms){
    unsigned int j, i;
    for(j=0; j< ms; j++){
        for(i = 0; i< 112; i++){
        }
    }
}

void command(int hex){
    RS = 0;
    LCD = hex;
    E = 1;
    delay(1);
    E = 0;
}

```

```

}

void setup_lcd(){
    unsigned int i;

    command(0x38);
    delay(2);

    command(0x0C);
    delay(2);

    command(0x01);
    delay(2);

    command(0x06);
    delay(2);

    for( i=0; i < 0x0C; i++){
        show_char(string[i]);
    }

    command(0xC0);
    delay(2);

    for( i=0; i < 0x04; i++){
        show_char(set[i]);
    }
    show_value(set_value);
}

void show_char(unsigned char x){
    RS = 1;
    LCD = x;
    E = 1;
    delay(10);
    E = 0;
}

unsigned char read_adc(){
    unsigned char val;

    WR_ADC = 0;
    WR_ADC = 1;

    while(INTR == 1);

    RD_ADC = 0;
    val = ADC_DATA;
    RD_ADC = 1;
}

```

```

    return val;
}

void process_data(unsigned char val){

    temp_value = ((unsigned int)val * 80 ) / 51;

    if(temp_value < (set_value - 5)){
        RELAY = 1;
    }
    else if( temp_value > (set_value + 5)){
        RELAY = 0;
    }

    show_value(temp_value);
}

void show_value(unsigned int val){
    unsigned char tens, ones, tenth;

    tens = val / 100;
    ones = (val % 100) / 10;
    tenth = val % 10;

    if (tens > 0){
        show_char(tens + '0');
    }
    else{
        show_char(' ');
    }

    show_char(ones + '0');
    show_char('.');
    show_char(tenth + '0');
}

```

```

del EQU 99h
ADC_DATA EQU P1
LCD EQU P2
RS EQU P0.0
RW EQU P0.1
E EQU P0.2
INTR EQU P3.0
BTN_PLUS EQU P3.2
BTN_MINUS EQU P3.3
RELAY EQU P3.5
WR_ADC EQU P3.6
RD_ADC EQU P3.7

SET_VAL_L EQU 30h
SET_VAL_H EQU 31h
TEMP_VAL_L EQU 32h
TEMP_VAL_H EQU 33h
FLAG_LCD BIT 20h.0

ORG 0000h
    LJMP MAIN

ORG 0003h
    LJMP ISR_PLUS

ORG 0013h
    LJMP ISR_MINUS

ORG 0050h
    text_temp: DB "Temperature:",0
    text_set: DB "Set:",0

ORG 0100h

MAIN:
    MOV ADC_DATA, #0FFh
    MOV LCD, #00h
    CLR RW
    CLR RELAY
    SETB INTR
    SETB BTN_PLUS
    SETB BTN_MINUS
    SETB IT0
    SETB IT1
    MOV IE, #85h

    MOV SET_VAL_H, #00h
    MOV SET_VAL_L, #0D2h

    ACALL SETUP_LCD

```

```

LOOP:
    CLR EA
    MOV A, #8Ch
    ACALL COMMAND
    ACALL READ_ADC
    ACALL PROCESS_DATA
    SETB EA

    JNB FLAG_LCD, NOT_FLAG
    CLR EA
    MOV A, #0C4h
    ACALL COMMAND

    MOV R2, SET_VAL_H
    MOV R3, SET_VAL_L

    ACALL SHOW_VALUE
    SETB EA
    CLR FLAG_LCD
NOT_FLAG:
    MOV A, #2Ch
    MOV B, #01h
    ACALL DELAY
    JMP LOOP

```

```

ISR_PLUS:
    PUSH PSW
    PUSH ACC
    PUSH B

    MOV A, #14h
    MOV B, #00h
    ACALL DELAY

    JB BTN_PLUS, EXIT_PLUS

    CLR C
    MOV A, SET_VAL_L
    SUBB A, #2Ch
    MOV A, SET_VAL_H
    SUBB A, #01h
    JNC WAIT_RELEASE_PLUS

    MOV A, SET_VAL_L
    ADD A, #05h
    MOV SET_VAL_L, A
    MOV A, SET_VAL_H
    ADDC A, #00h
    MOV SET_VAL_H, A

```

```

SETB FLAG_LCD

WAIT_RELEASE_PLUS: JNB BTN_PLUS, WAIT_RELEASE_PLUS

MOV A, #14h
MOV B, #00h
ACALL DELAY

EXIT_PLUS:
POP B
POP ACC
POP PSW
RETI

ISR_MINUS:
PUSH PSW
PUSH ACC
PUSH B

MOV A, #14h
MOV B, #00h
ACALL DELAY

JB BTN_MINUS, EXIT_MINUS

CLR C
MOV A, SET_VAL_L
SUBB A, #65h
MOV A, SET_VAL_H
SUBB A, #00h
JC WAIT_RELEASE_MINUS

CLR C
MOV A, SET_VAL_L
SUBB A, #05h
MOV SET_VAL_L, A
MOV A, SET_VAL_H
SUBB A, #00h
MOV SET_VAL_H, A

SETB FLAG_LCD

WAIT_RELEASE_MINUS: JNB BTN_MINUS, WAIT_RELEASE_MINUS

MOV A, #14h
MOV B, #00h
ACALL DELAY

EXIT_MINUS:

```

```

    POP B
    POP ACC
    POP PSW
RETI

DELAY:
    PUSH 07
    PUSH ACC
    ORL A, B
    JNZ OK

    POP ACC
    POP 07
    RET
OK:
    POP ACC
DEC_LSB:
    ACALL ONEMIL
    DJNZ ACC, DEC_LSB

    CJNE A, B, DEC_MSB
    POP 07
    RET
DEC_MSB:
    DEC B

    SJMP DEC_LSB

ONEMIL:
    MOV R7, #del
    TIMER:
        NOP
        NOP
        NOP
        NOP
        DJNZ R7, TIMER
        NOP
RET

SETUP_LCD:

    MOV A, #38h
    ACALL COMMAND

    MOV A, #0Ch
    ACALL COMMAND

    MOV A, #01h
    ACALL COMMAND

```

```

MOV A, #06h
ACALL COMMAND

MOV DPTR, #text_temp
SHOW_TEXT_1:
    CLR A
    MOVC A, @A+DPTR
    JZ END_PRINT_1
    ACALL SHOW_CHAR
    INC DPTR
    JMP SHOW_TEXT_1
END_PRINT_1:

```

```

MOV A, #0C0h
ACALL COMMAND

```

```

MOV DPTR, #text_set
SHOW_TEXT_2:
    CLR A
    MOVC A, @A+DPTR
    JZ END_PRINT_2
    ACALL SHOW_CHAR
    INC DPTR
    JMP SHOW_TEXT_2
END_PRINT_2:
    MOV R2, SET_VAL_H
    MOV R3, SET_VAL_L
    ACALL SHOW_VALUE

```

```

RET

```

```

COMMAND:
    CLR RS
    MOV LCD, A
    SETB E
    ACALL ONEMIL
    ACALL ONEMIL
    CLR E

```

```

RET

```

```

SHOW_CHAR:
    SETB RS
    MOV LCD, A
    SETB E
    MOV A, #0Ah
    MOV B, #00h
    ACALL DELAY
    CLR E

```

```

RET

```

```

READ_ADC:

```



```

CLR WR_ADC
NOP
NOP
SETB WR_ADC

WAIT: JB INTR, WAIT
CLR RD_ADC
NOP
NOP
MOV A, ADC_DATA
SETB RD_ADC
RET

PROCESS_DATA:
MOV B, #50h
MUL AB
MOV R0, B
MOV R1, A

MOV R2, #00h
MOV R3, #00h

DIV_51:
CLR C

MOV A, R1
SUBB A, #33h
MOV R1, A

MOV A, R0
SUBB A, #00h
MOV R0, A

JC DIV_DONE
INC R3
MOV A, R3
JNZ DIV_51
INC R2
JMP DIV_51
DIV_DONE:
MOV TEMP_VAL_H, R2
MOV R0, TEMP_VAL_H
MOV TEMP_VAL_L, R3
MOV R1, TEMP_VAL_L

VERIFY_1:
CLR C
MOV A, SET_VAL_L
SUBB A, #05h
MOV R4, A

```

```

MOV A, SET_VAL_H
SUBB A, #00h
MOV R5, A

CLR C
MOV A, TEMP_VAL_L
SUBB A, R4
MOV A, TEMP_VAL_H
SUBB A, R5
JNC VERIFY_2

SETB RELAY
SJMP FINISH
VERIFY_2:
MOV A, SET_VAL_L
ADD A, #05h
MOV R4, A
MOV A, SET_VAL_H
ADDC A, #00h
MOV R5, A

CLR C
MOV A, TEMP_VAL_L
SUBB A, R4
MOV A, TEMP_VAL_H
SUBB A, R5
JC FINISH

CLR RELAY
FINISH:
ACALL SHOW_VALUE
RET

SHOW_VALUE:
MOV R4, #00h
MOV R5, #00h
MOV R6, #00h

LOOP_TENS:
CLR C

MOV A, R3
SUBB A, #64h
MOV R3, A

MOV A, R2
SUBB A, #00h
MOV R2, A

JC LOOP_TENS_DONE

```

```

    INC R4
    JMP LOOP_TENS
LOOP_TENS_DONE:
    MOV A, R3
    ADD A, #64h
    MOV R3, A

    ONES_AND_TENTH:
    MOV A, R3
    MOV B, #0Ah
    DIV AB
    MOV R5, A
    MOV R6, B
PRINT_TEMP:
    MOV A, R4
    JZ SPACE
    ADD A, #30h
    JMP PRINT
SPACE:
    MOV A, #20h
PRINT:
    ACALL SHOW_CHAR

    MOV A, R5
    ADD A, #30h
    ACALL SHOW_CHAR

    MOV A, #2Eh
    ACALL SHOW_CHAR

    MOV A, R6
    ADD A, #30h
    ACALL SHOW_CHAR
RET
END

```